

南京大学本科期末考试试卷（模拟·A 卷）

—— 依据 2025-2026 学年真题题型与考点定制·全真模拟 ——

课程名称：《高级程序设计》

姓名：_____

学年学期：2025-2026 学年 第 1 学期

学号：_____

开课单位：智能软件与工程学院

年级：_____

考试方式：闭卷

专业：_____

考试时长：120 分钟；总分 100 分

题号	一	二	三	总分
得分				

一、简答题（共 4 题，共 20 分）

得分	
----	--

请简明扼要作答，写出定义 + 区别 / 条件 / 作用，要点齐全即可。

1. 函数重写（override）与函数重载（overload）有什么区别？请各举一个简单例子说明。（5 分）

2. 什么是纯虚函数？什么是抽象类？抽象类有什么作用？在 C++ 中如何定义纯虚函数？（5 分）

3. 深拷贝与浅拷贝有什么区别？在什么情况下必须为一个类自定义拷贝构造函数（实现深拷贝）？（5 分）

4. 什么是友元（friend）函数？它与类的成员函数有什么区别？为什么 C++ 要引入友元机制？（5 分）

二、程序分析题（共 4 题，共 40 分）

得分	
----	--

请写出必要的分析过程。若出现未定义行为（Undefined Behavior）导致输出不确定，请将不确定的结果用 xxx 代替。

（说明：返回局部对象、值传参等场景按笔试惯例分析，即等价于编译选项 `-fno-elide-constructors`，return 局部对象计为一次拷贝构造。）

1. (10 分) 阅读下面程序, 回答问题。

代码	问题
<pre> #include <iostream> #include <string> using namespace std; class Engine { string type; public: Engine(string t) : type(t) { cout << "Engine(" << type << ") ctor" << endl; } Engine(const Engine& e) : type(e.type) { cout << "Engine(" << type << ") copy" << endl; } ~Engine() { cout << "Engine(" << type << ") dtor" << endl; } }; class Vehicle { string name; public: Vehicle(string n) : name(n) { cout << "Vehicle(" << name << ") ctor" << endl; } Vehicle(const Vehicle& v) : name(v.name) { cout << "Vehicle(" << name << ") copy" << endl; } ~Vehicle() { cout << "Vehicle(" << name << ") dtor" << endl; } }; class Car : public Vehicle { Engine engine; public: Car(string n, string t) : Vehicle(n), engine(t) { cout << "Car(" << n << ") ctor" << endl; } Car(const Car& c) : Vehicle(c), engine(c.engine) { cout << "Car copy" << endl; } ~Car() { cout << "Car dtor" << endl; } }; Car makeCar() { Car c("Tesla", "V8"); return c; } void test(Car c) { cout << "-- in test --" << endl; } int main() { cout << "===1===" << endl; Car c1("BMW", "L4"); cout << "===2===" << endl; Car c2 = makeCar(); cout << "===3===" << endl; test(c1); cout << "===4===" << endl; return 0; } </pre>	1. 写出程序的完整输出。 2. 指出程序中所有调用拷贝构造函数的位置; Car c2 = makeCar(); 这条语句本身是否会再额外调用一次拷贝构造? 3. 如果删除 Car 类中自定义的拷贝构造函数 (改用编译器默认生成的), 程序还能正常运行吗? 输出会有什么变化? 为什么?

2. (10 分) 阅读下面程序, 回答问题。

代码	问题
<pre> #include <iostream> using namespace std; class Animal { public: virtual void sound() { cout << "Animal: ..." << endl; } void intro() { cout << "I say: "; sound(); } virtual ~Animal() {} }; class Dog : public Animal { public: void sound() override { cout << "Dog: Woof" << endl; } }; class Cat : public Animal { public: void sound() override { cout << "Cat: Meow" << endl; } }; int main() { Dog d; Cat c; Animal* arr[2] = { &d, &c }; for (int i = 0; i < 2; i++) arr[i]->intro(); Animal* p = new Dog(); p->sound(); delete p; return 0; } </pre>	1. 写出程序的运行结果。 2. intro() 并不是虚函数, 为什么它内部调用 sound() 时能根据实际对象分别输出 Dog / Cat 的声音? 请用正确术语解释。 3. 如果去掉 Animal::sound() 前的 virtual (相应 override 也一并去掉), 通过基类指针 arr[i]->intro() 的输出会变成什么? 为什么?

3. (10 分) 阅读下面程序，回答问题。

代码	问题
<pre>#include <iostream> using namespace std; class Vector2D { double x, y; public: Vector2D(double x = 0, double y = 0) : x(x), y(y) {} Vector2D operator+(const Vector2D& v) const { return Vector2D(x + v.x, y + v.y); } Vector2D& operator++() { ++x; ++y; return *this; } bool operator==(const Vector2D& v) const { return x == v.x && y == v.y; } friend ostream& operator<<(ostream& os, const Vector2D& v) { os << "(" << v.x << ", " << v.y << ")"; return os; } }; int main() { Vector2D a(1, 2), b(3, 4); Vector2D c = a + b; cout << "c = " << c << endl; cout << "a == b ? " << boolalpha << (a == b) << endl; Vector2D d = ++a; cout << "a = " << a << ", d = " << d << endl; cout << "a + b == c ? " << (a + b == c) << endl; return 0; }</pre>	1. 写出程序的运行结果。 2. <code>cout << "c = " << c << endl;</code> 为什么能正确执行？如果把 <code>operator<<</code> 的返回类型由 <code>ostream&</code> 改成 <code>void</code>，会发生什么？ 3. 现在要支持 <code>a * 2.0</code>（<code>Vector2D</code> 各分量乘以一个 <code>double</code> 标量），请写出对应的运算符重载函数。

4. (10 分) 阅读下面程序，回答问题。

代码	问题
<pre>#include <iostream> #include <stdexcept> using namespace std; int stage = 0; void inner(int x) { stage = 1; if (x == 0) throw runtime_error("divide by zero"); stage = 2; cout << "inner: " << 100 / x << endl; } void middle(int x) { stage = 3; try { inner(x); stage = 4; cout << "middle: ok" << endl; } catch (const exception& e) { stage = 5; cout << "middle caught: " << e.what() << endl; } stage = 6; cout << "middle: done" << endl; } int main() { try { stage = 0; middle(0); stage = 7; cout << "main: ok" << endl; } catch (const exception& e) { stage = 8; cout << "main caught: " << e.what() << endl; } cout << "Final stage = " << stage << endl; return 0; }</pre>	1. 写出程序的完整输出，并给出 <code>stage</code> 的最终值。分析 <code>middle</code> 中 <code>stage = 4;</code> 这行是否会被执行？为什么？ 2. 如果在 <code>middle</code> 的 <code>catch</code> 块末尾添加语句 <code>throw;</code>，程序的输出和 <code>stage</code> 最终值会怎样变化？请解释异常传播（栈展开）机制的改变。

三、编程题（共 2 题，共 40 分）

得分

1. (20 分) 复数类 `Complex` —— 构造、加减运算、相等比较与输出。

- 根据 `//TODO` 注释，在类外补全 `Complex` 的 5 个成员/友元函数实现。
- 复数以“实部 + 虚部 `i`”形式表示；构造函数支持 `Complex()`、`Complex(5)`、`Complex(3,4)` 三种调用。

- 重载 +、-（返回新对象）、==（返回 bool）、<<（friend，支持链式输出，虚部为负时用减号）。

需要补充的代码（类定义）：

```
#include <iostream>
using namespace std;

class Complex {
private:
    double re;    // 实部
    double im;    // 虚部
public:
    // TODO 1: 构造函数（支持 Complex()、Complex(5)、Complex(3,4)）
    Complex(double r = 0, double i = 0);

    // TODO 2: 加法 (a+bi)+(c+di) = (a+c)+(b+d)i
    Complex operator+(const Complex& rhs) const;

    // TODO 3: 减法 (a+bi)-(c+di) = (a-c)+(b-d)i
    Complex operator-(const Complex& rhs) const;

    // TODO 4: 相等比较（实部、虚部均相等返回 true）
    bool operator==(const Complex& rhs) const;

    // TODO 5: 输出流（虚部为负时用减号，如 3-4i；其余如 3+4i、0+0i、5+0i）
    friend ostream& operator<<(ostream& os, const Complex& c);
};
```

你的实现应能通过以下测试：

```
int main() {
    Complex a(3, 4);    // 3+4i
    Complex b(1, -2);   // 1-2i
    Complex c;          // 0+0i
    Complex d(5);       // 5+0i
    cout << a << endl;   cout << b << endl;
    cout << c << endl;   cout << d << endl;

    Complex e = a + b;   cout << e << endl;    // 4+2i
    Complex f = a - b;   cout << f << endl;    // 2+6i

    cout << boolalpha << (a == Complex(3, 4)) << endl; // true
    cout << (a == b) << endl;                       // false

    Complex g = a + b - d; cout << g << endl;      // -1+2i
    return 0;
}
```

2. (20 分) 引用计数智能指针 RefPtr 与链表容器 DataPool。

- 根据注释补全 RefPtr（12 分）与 DataPool（8 分）。
- RefPtr 需正确维护引用计数，计数归零时释放对象；DataPool 内部是单向链表，每个节点存一个 RefPtr。
- 只能在已有框架内补充代码，不得删除或修改原有结构；main 已给出。

需要补充的代码：

```
#include <iostream>
#include <functional>
#include <vector>
#include <cassert>
#include <string>
using namespace std;

template<typename T> class DataPool;

// 引用计数控制块
struct Counter { int n; Counter() : n(1) {} };
```

```

// TODO 1: 引用计数智能指针 RefPtr (12 分)
template<typename T>
class RefPtr {
    friend class DataPool<T>;
private:
    T* ptr;
    Counter* cnt;
    void release() { // 已实现: 引用计数减一, 归零则释放
        if (cnt) {
            cnt->n--;
            if (cnt->n == 0) {
                delete ptr; delete cnt;
                cout << "[freed]" << endl;
            }
        }
        ptr = nullptr; cnt = nullptr;
    }
public:
    RefPtr() : ptr(nullptr), cnt(nullptr) {}
    explicit RefPtr(T* p) : ptr(p) { cnt = p ? new Counter() : nullptr; }

    // TODO 1.1 拷贝构造: 浅拷贝指针, 引用计数加一
    RefPtr(const RefPtr& other) {

    }

    // TODO 1.2 析构函数
    ~RefPtr() {

    }

    // TODO 1.3 赋值运算符
    // 提示: 自赋值检测、释放旧资源、接管新资源并加一、return *this
    RefPtr& operator=(const RefPtr& other) {

    }

    // TODO 1.4 解引用: ptr 为空则断言失败, 否则返回 *ptr
    T& operator*() const {

    }

    T* operator->() const { return ptr; }
    int use_count() const { return cnt ? cnt->n : 0; }
    bool valid() const { return ptr != nullptr; }
};

// 单向链表节点
template<typename T>
struct PoolNode {
    RefPtr<T> data;
    PoolNode* next;
    PoolNode(const RefPtr<T>& d) : data(d), next(nullptr) {}
};

// TODO 2: 链表容器 DataPool (8 分)
template<typename T>
class DataPool {
private:
    PoolNode<T>* head;
    size_t count;
public:
    DataPool() : head(nullptr), count(0) {}
    ~DataPool() {
        PoolNode<T>* cur = head;
        while (cur) { PoolNode<T>* nxt = cur->next; delete cur; cur = nxt; }
    }
    DataPool(const DataPool&) = delete;
    DataPool& operator=(const DataPool&) = delete;

    // TODO 2.1 登记新对象: 构造 RefPtr, new 节点, 头插
    void add(T* raw) {

    }

    // TODO 2.2 map_to: 把每个对象映射为 U 类型, 收集进 vector<U>
    template<typename U, typename Func>
    vector<U> map_to(Func f) const {
        vector<U> result;
        // 在此补充
    }

```

```

        return result;
    }

    // TODO 2.3 select: 收集满足 pred 的对象副本
    vector<T> select(function<bool(const T&> pred) const {
        vector<T> result;
        // 在此补充
        return result;
    })

    void show() const {
        PoolNode<T>* cur = head; cout << "[ ";
        while (cur) { if (cur->data.valid()) cout << *(cur->data) << " "; cur = cur->next; }
        cout << "]" << endl;
    }
};

```

你的实现应能通过以下测试：

```

class Item {
public:
    string tag; int val;
    Item(string t, int v) : tag(t), val(v) { cout << "Item " << tag << " born" << endl; }
    ~Item() { cout << "Item " << tag << " gone" << endl; }
    friend ostream& operator<<(ostream& os, const Item& it) {
        os << "{" << it.tag << ":" << it.val << "}"; return os;
    }
};

int main() {
    {
        RefPtr<Item> p1(new Item("X", 1));
        RefPtr<Item> p2 = p1;
        cout << "count = " << p1.use_count() << endl;    // 2
        RefPtr<Item> p3;
        p3 = p2;
        cout << "count = " << p1.use_count() << endl;    // 3
        cout << "access: " << p3->tag << endl;            // X
    }
    cout << "-----" << endl;

    DataPool<Item> pool;
    pool.add(new Item("A", 3));
    pool.add(new Item("B", 8));
    pool.add(new Item("C", 5));
    pool.show();                                          // [ {C:5} {B:8} {A:3} ]

    auto nums = pool.map_to<int>([](const Item& it){ return it.val * 10; });
    cout << "map: ";
    for (auto n : nums) cout << n << " ";                // 50 80 30
    cout << endl;

    auto big = pool.select([](const Item& it){ return it.val >= 5; });
    cout << "select: ";
    for (auto& it : big) cout << it << " ";              // {C:5} {B:8}
    cout << endl;
    return 0;
}

```